

JAVA

Generics em Java

Classes Genéricas • Bounds • Wildcards • Type Erasure • Comparable

Apostila completa do básico ao avançado,
com exemplos práticos e mini-projects guiados.

CONTEÚDO DA APOSTILA

- 01 Introdução — o problema que os Generics resolvem
- 02 Criando Classes e Métodos Genéricos
- 03 Convenções T, E, K, V e Diamond Operator
- 04 Limites Superiores (Bounds) com extends
- 05 Múltiplos Bounds e Métodos Estáticos
- 06 Tipos de Referência Genéricos — a armadilha
- 07 Wildcards: `<?>`, `<? extends T>`, `<? super T>`
- 08 PECS e Type Erasure
- 09 Comparable e Comparator
- 10 Generics no Java Moderno (8–21)
- 11 Padrões de Projeto com Generics
- 12 Mini-Projects: Layer, QueryList, Repository

1

Introdução aos Generics

O problema do mundo sem tipos, segurança em compilação e `ArrayList<T>`

Antes dos Generics, escrever código reutilizável em Java significava abrir mão da segurança de tipos. Esta apostila explica por que os Generics foram criados, como usá-los do zero, e como os padrões modernos do Java — Streams, Records, Lambdas — dependem diretamente deles.

O problema antes dos Generics (pré-Java 5)

Antes do Java 5 (2004), as coleções da JDK armazenavam apenas `Object`. Isso significava que qualquer tipo podia ser inserido em qualquer coleção — sem verificação alguma em tempo de compilação. Para recuperar um elemento, o programador precisava fazer um cast manual, correndo o risco de uma `ClassCastException` em tempo de execução, um erro difícil de rastrear em sistemas maiores.

O Java introduziu os Generics exatamente para resolver esse problema: mover a verificação de tipo do *runtime* para o *compile-time*. Com Generics, o compilador sabe exatamente que tipo a coleção armazena, rejeita inserções inválidas antes do programa rodar, e elimina a necessidade de casting manual.

```
// 🚦 ANTES dos Generics – código frágil:
List lista = new ArrayList();           // sem tipo
lista.add("texto");
lista.add(42);                          // mistura tipos! ninguém avisa.
String s = (String) lista.get(0);       // cast manual
String err = (String) lista.get(1);     // ClassCastException em RUNTIME!

// ■ COM Generics – seguro e limpo:
List<String> lista = new ArrayList<>();
lista.add("texto");
// lista.add(42); → ERRO DE COMPILAÇÃO – o compilador protege você!
String s = lista.get(0);                 // sem cast – tipo garantido
```

■ Custo zero em runtime

Generics são verificados em tempo de compilação e removidos do bytecode (type erasure). O custo é zero em runtime — toda a segurança é gratuita.

2

Criando Classes Genéricas

Sintaxe, parâmetros de tipo, Diamond Operator e Raw Types

Declarando uma Classe Genérica

Para tornar uma classe genérica, adiciona-se o parâmetro de tipo entre `< >` logo após o nome da classe. O identificador mais comum é **T** (de *Type*), mas pode ser qualquer nome válido. Dentro da classe, esse identificador é usado como se fosse um tipo real — o compilador substitui **T** pelo tipo concreto cada vez que a classe é usada.

A classe genérica é um molde parametrizado: `Box<String>` e `Box<Integer>` são duas instâncias diferentes do mesmo molde, cada uma com comportamento de tipos distintos, sem duplicação de código. Isso é a essência da reutilização genérica.

```
// Classe comum – limitada a um tipo:
class Caixa { Object conteudo; }

// Classe genérica – funciona para qualquer tipo:
class Box<T> {
    private T conteudo;
    public Box(T conteudo) { this.conteudo = conteudo; }
    public T getConteudo() { return conteudo; }
    public void setConteudo(T conteudo) { this.conteudo = conteudo; }
    @Override
    public String toString() { return "Box[" + conteudo + "]"; }
}

// Uso – T é substituído pelo tipo concreto:
Box<String> caixaTexto = new Box<>("Olá!"); // T = String
Box<Integer> caixaNum = new Box<>(42); // T = Integer
System.out.println(caixaTexto.getConteudo()); // "Olá!"
System.out.println(caixaNum.getConteudo() * 2); // 84
```

Convenções de Nomenclatura e Diamond Operator

A convenção da JDK para nomes de parâmetros de tipo usa letras únicas maiúsculas: **T** (Type) para tipos genéricos gerais; **E** (Element) para elementos de coleções; **K** e **V** (Key, Value) para mapas; **N** (Number) para tipos numéricos. Com múltiplos parâmetros, usa-se **S**, **U**, **V** para o segundo, terceiro e quarto respectivamente.

O **Diamond Operator** `<>`, introduzido no Java 7, permite que o compilador infira o argumento de tipo a partir da declaração da variável, evitando a repetição verbosa. Antes: `new ArrayList<String>()`; depois:

`new ArrayList<>()`. O resultado é idêntico.

```
// Convenções de nomenclatura:
class Lista<E> { List<E> items; }           // E = Element
class Mapa<K,V> { Map<K,V> entries; }      // K/V = Key/Value
class Par<A,B> { A primeiro; B segundo; }   // múltiplos params

// Diamond Operator – antes vs depois:
List<String> antes = new ArrayList<String>(); // verboso
List<String> depois = new ArrayList<>();      // Diamond – ■ preferido

// ■ Raw Type – evite em código novo!
List rawList = new ArrayList(); // sem tipo – pré-Java 5
rawList.add("a"); rawList.add(42); // sem verificação
String s = (String) rawList.get(1); // ClassCastException!
```

■ ■ Nunca use Raw Types em código novo

Raw Types (List sem <T>) existem apenas por retrocompatibilidade com código pré-Java 5. Em código novo, especifique sempre o tipo — o compilador emitirá warnings de 'unchecked' para Raw Types.

3

Limites Superiores (Bounds)

extends, múltiplos bounds e métodos estáticos genéricos

Por que limitar o tipo genérico?

Sem limite, um parâmetro de tipo aceita qualquer coisa — inclusive tipos que não fazem sentido para a lógica da classe. Um limite superior (**upper bound**) restringe os tipos aceitos usando `extends`: `T extends Jogador` garante que qualquer `T` usado nessa classe é `Jogador` ou um subtipo dele.

Essa restrição traz dois benefícios: segurança (só tipos compatíveis podem ser usados) e acesso (dentro da classe genérica, você pode chamar qualquer método de `Jogador` em uma variável do tipo `T`, porque o compilador garante que `T` implementa esses métodos). Note que `extends` aqui **NÃO** significa herança de classe — significa 'é do tipo ou subtipo/implementação de'.

```
// Interface como bound:
interface Jogador {
    String getNome();
    int getNumero();
}

// T deve ser Jogador ou subtipo – garante acesso a getNome():
class Time<T extends Jogador> {
    private List<T> membros = new ArrayList<>();

    public void adicionar(T membro) { membros.add(membro); }

    public void listar() {
        // Podemos chamar getNome() porque o bound garante!
        membros.forEach(m -> System.out.println(
            m.getNome() + " #" + m.getNumero()));
    }
}

// Uso:
Time<Atacante> timeAtacantes = new Time<>(); // Atacante extends Jogador
Time<String> errado = new Time<>();          // ■ String não é Jogador
```

Múltiplos Bounds e Métodos Estáticos Genéricos

É possível exigir que T satisfaça múltiplas restrições simultaneamente, separadas por &. A regra é: no máximo uma **classe** (que deve vir primeiro), seguida de zero ou mais **interfaces**. O tipo resultante precisa satisfazer todas as restrições ao mesmo tempo.

Métodos estáticos não podem usar o parâmetro de tipo da classe (pois não têm instância). A solução é declarar um parâmetro de tipo próprio, diretamente no método, *antes* do tipo de retorno: static <E> Container<E> criar(E e). O T do método é independente do T da classe.

```
// Múltiplos bounds – classe primeiro, interfaces depois:
class Repositorio<T> extends EntidadeBase & Comparable<T> & Serializable<> {
    private List<T> items = new ArrayList<>();
    public void salvar(T item) { items.add(item); }
    public List<T> ordenados() {
        return items.stream().sorted().toList(); // usa Comparable!
    }
}

// Método estático com parâmetro de tipo PRÓPRIO:
class Container<T> {
    private T valor;
    // ■ static não acessa T da classe:
    // static T criar() { return null; }

    // ■ declara E no próprio método:
    static <E> Container<E> criar(E elemento) {
        Container<E> c = new Container<>();
        c.valor = elemento;
        return c;
    }
}
```

4

Tipos de Referência Genéricos

Por que `List<Filho>` NÃO é subtipo de `List<Pai>` — e como resolver

A armadilha da 'herança' entre coleções genéricas

É natural assumir que, se `LPASStudent` herda de `Student`, então `List<LPASStudent>` poderia ser atribuída a `List<Student>`. Essa intuição está errada — e o Java proíbe isso intencionalmente por segurança de tipo.

Se fosse permitido, o código poderia adicionar um `Student` comum a uma lista que, em tempo de execução, é de `LPASStudent` — corrompendo a integridade da coleção. O que funciona é a herança da implementação: `ArrayList<String>` pode ser atribuído a `List<String>` porque `ArrayList` implementa `List` — mas o parâmetro de tipo `String` deve ser idêntico.

```
// Herança de classe – FUNCIONA normalmente:
LPASStudent aluno = new LPASStudent();
Student s = aluno;           // OK – LPASStudent IS-A Student

// Herança de ArrayList – FUNCIONA (implementação):
List<String> l = new ArrayList<>(); // ArrayList implements List

// ■ Herança entre GENÉRICOS – NÃO funciona:
List<LPASStudent> lpas = new ArrayList<>();
// List<Student> students = lpas; // ERRO DE COMPILAÇÃO!

// POR QUÊ? Se fosse permitido:
// students.add(new Student()); // Student comum na lista de LPASStudent
// LPASStudent x = lpas.get(0); // ClassCastException em runtime!

// ■ SOLUÇÃO – use wildcards:
List<? extends Student> view = lpas; // leitura segura
```

LPASStudent → Student	■ Sim	Herança de classe normal (IS-A)
ArrayList<T> → List<T>	■ Sim	ArrayList implementa List (mesmo T)
List<LPASStudent> → List<Student>	■ Não	Tipos genéricos não são covariantes
List<? extends Student>	■ Sim (leitura)	Wildcard com upper bound resolve
List<? super LPASStudent>	■ Sim (escrita)	Wildcard com lower bound resolve

5

Wildcards, PECS e Type Erasure

<?>, <? extends T>, <? super T>, regra PECS e apagamento de tipo

Os 3 tipos de Wildcard

Um **wildcard** (?) representa um tipo desconhecido em um argumento de tipo genérico. É usado *exclusivamente em argumentos de tipo* — nunca na declaração do parâmetro de tipo de uma classe ou método genérico. Há três variantes, cada uma com seu caso de uso específico.

O **wildcard irrestrito** <?> aceita qualquer tipo — útil quando você precisa processar uma lista genérica sem saber ou importar qual é o tipo concreto. O **upper bound** <? extends T> aceita T ou qualquer subtipo de T — ideal para *leitura* de dados. O **lower bound** <? super T> aceita T ou qualquer supertipo de T — ideal para *escrita* de dados.

```
// 1. Wildcard irrestrito – aceita qualquer List:
void imprimirTodos(List<?> lista) {
    for (Object item : lista) System.out.println(item);
}

// 2. Upper bound – lê dados de List de Number ou subtipo:
double somarNumeros(List<? extends Number> numeros) {
    return numeros.stream().mapToDouble(Number::doubleValue).sum();
    // Aceita: List<Integer>, List<Double>, List<Float>...
}

// 3. Lower bound – escreve Integer ou supertipo em List:
void adicionarInteiros(List<? super Integer> lista, int qtd) {
    for (int i = 1; i <= qtd; i++) lista.add(i);
    // Aceita: List<Integer>, List<Number>, List<Object>
}
```

Regra PECS e Type Erasure

A regra **PECS** (Producer Extends, Consumer Super) é um mnemônico para lembrar qual wildcard usar: se a coleção *produz* dados para você ler, use `<? extends T>`; se a coleção *consome* dados que você escreve, use `<? super T>`. Uma coleção que faz os dois deve usar um parâmetro de tipo regular (sem wildcard).

Type Erasure é o mecanismo pelo qual o compilador remove os Generics do bytecode. Após a compilação, `T` sem bound vira `Object`; `T extends Number` vira `Number`. Isso significa que `List<String>` e `List<Integer>` são indistinguíveis em runtime — ambas são apenas `List` no bytecode. Por isso, dois métodos sobrecarregados que diferem apenas no parâmetro genérico causam erro de compilação após o erasure.


```
// PECS na prática – copiar de src para dest:
<T> void copiar(List<? extends T> src,    // Producer: extends (lê)
               List<? super T> dest) { // Consumer: super (escreve)
    for (T item : src) dest.add(item);
}

// Type Erasure – colisão de assinatura:
// ■ ERRO – ambos viram process(List) após erasure:
// void process(List<String> l) { }
// void process(List<Integer> l) { } // conflito!

// Type Erasure – bytecode de Box<T>:
// class Box<T> { T item; } → class Box { Object item; }
// class Num<T extends Number> { } → class Num { Number t; }

// ■ Não se pode instanciar T diretamente:
// T obj = new T(); // ERRO de compilação
// ■ Use Supplier:
<T> T criar(java.util.function.Supplier<T> factory) {
    return factory.get(); // ex: () -> new MinhaClasse()
}
```

6

Comparable & Comparator

Ordenação natural, alternativa e `Comparator.comparing()`

A interface Comparable — ordem natural

`Comparable<T>` é uma interface genérica da JDK com um único método: `compareTo(T outro)`. Implementá-la na sua classe define a **ordem natural** — a ordem padrão usada por `Collections.sort()`, `Arrays.sort()`, e estruturas ordenadas como `TreeSet`.

O contrato do `compareTo`: retornar um valor negativo se `this` é menor que o argumento, zero se são iguais, e positivo se `this` é maior. A boa prática é usar os métodos estáticos das wrapper classes: `Integer.compare(a, b)` ao invés de `a - b` (que pode causar overflow).

```

class Aluno implements Comparable<Aluno> {
    private int id;
    private String nome;
    private String curso;
    private double percentualConcluido;

    @Override
    public int compareTo(Aluno outro) {
        // Ordem natural: por ID crescente
        return Integer.compare(this.id, outro.id);
    }

    // Comparators alternativos como campos estáticos:
    public static Comparator<Aluno> POR_NOME =
        Comparator.comparing(a -> a.nome);

    public static Comparator<Aluno> POR_CURSO_DEPOIS_NOME =
        Comparator.comparing(Aluno::getCurso)
            .thenComparing(Aluno::getNome);
}

// Uso:
List<Aluno> turma = new ArrayList<>(/* ... */);
Collections.sort(turma); // ordem natural (por ID)
turma.sort(Aluno.POR_NOME); // por nome
turma.sort(Aluno.POR_CURSO_DEPOIS_NOME); // por curso, depois nome

```

Onde é implementado	Na própria classe	Classe separada / lambda / field estático
Método principal	compareTo(T outro)	compare(T a, T b)
Tipo de ordem	Natural — única	Alternativa — múltiplas
API de uso	Collections.sort(list)	list.sort(comp) ou Arrays.sort(arr, comp)
Java 8+	Ainda relevante	Comparator.comparing() + thenComparing()

7

Generics no Java Moderno & Padrões

Java 8–21, Streams, Records, Repository Pattern e erros comuns

Generics no Java Moderno (8–21)

Generics são o alicerce de praticamente tudo no Java moderno. O **Streams API** (Java 8) usa Generics extensivamente: `Stream<T>`, `Optional<T>`, `Function<T,R>`, `Predicate<T>`. O **var** (Java 10) infere o tipo genérico automaticamente. **Records** (Java 16) podem ser genéricos com uma sintaxe elegante.

```
// Java 8 – Streams com Generics:
Stream<String> nomes = lista.stream()
    .filter(s -> s.length() > 3)    // Predicate<String>
    .map(String::toUpperCase);      // Function<String,String>

// Java 9 – List.of() retorna genérico imutável:
List<String> fixo = List.of("a", "b", "c");

// Java 10 – var infere tipo genérico:
var nomes2 = new ArrayList<String>(); // ArrayList<String> inferido

// Java 16 – Records genéricos:
record Par<A, B>(A primeiro, B segundo) { }
Par<String, Integer> p = new Par<>("idade", 25);
System.out.println(p.primeiro()); // "idade"

// Repository Pattern genérico:
interface Repository<T, ID> {
    Optional<T> findById(ID id);
    List<T> findAll();
    void save(T entity);
    void delete(ID id);
}
class AlunoRepository implements Repository<Aluno, Integer> {
    // implementação concreta type-safe
}
```

Erros Comuns — Evite!

```
// ■ Instanciar tipo genérico diretamente:
// T obj = new T(); // ERRO – não sabe qual construtor chamar
// ■ Use Supplier ou reflexão:
<T> T criar(Supplier<T> factory) { return factory.get(); }

// ■ Array de tipo genérico:
// T[] arr = new T[10]; // ERRO de compilação
// ■ Cast com aviso suprimido (use com cautela):
@SuppressWarnings("unchecked")
T[] arr = (T[]) new Object[10];

// ■ Wildcard na instanciação:
// new ArrayList<?>() // ERRO – não se instancia com wildcard

// ■ Dois métodos que colidem após type erasure:
// void process(List<String> l) { }
// void process(List<Integer> l) { } // conflito!
```

8

Mini-Projects

Classe `Layer<T>`, `QueryList` com `Comparable`, e `Repository Pattern`

Os mini-projects aplicam os conceitos desta apostila em cenários progressivos. O primeiro cria um sistema de mapeamento com classe genérica limitada. O segundo integra Generics com herança e `Comparable`. O terceiro aplica o `Repository Pattern` genérico.

Mini-Project A

Sistema de Mapeamento — Layer

Crie a interface `Mappable` com o método `render()`. Implemente as classes abstratas `Point` (latitude, longitude) e `Line` (array 2D de coordenadas). Crie `Park` (extends `Point`) e `River` (extends `Line`), cada um com campo `nome`. A classe genérica `Layer` deve ter uma lista privada, `addElement(T)` e `renderLayer()` que chama `render()` em cada elemento.

```
interface Mappable { void render(); }

abstract class Point implements Mappable {
    private double lat, lng;
    public Point(double lat, double lng) { this.lat=lat; this.lng=lng; }
    @Override public void render() {
        System.out.printf("POINT(%.4f, %.4f)%n", lat, lng);
    }
}

class Park extends Point {
    private String nome;
    public Park(String nome, double lat, double lng) {
        super(lat, lng); this.nome = nome;
    }
    @Override public void render() {
        System.out.print("Park: " + nome + " → "); super.render();
    }
}

class Layer<T extends Mappable> {
    private List<T> elementos = new ArrayList<>();
    public void addElement(T el) { elementos.add(el); }
    public void renderLayer() { elementos.forEach(Mappable::render); }
}

// Uso:
Layer<Park> parkLayer = new Layer<>();
parkLayer.addElement(new Park("Ibirapuera", -23.587, -46.657));
parkLayer.renderLayer();
```

Mini-Project B

QueryList — Comparable + ArrayList

Faça `QueryList` estender `ArrayList`. Implemente `Student` com campo `id` e `Comparable` por `id`. `LPASStudent` sobrescreve `matchFieldValue` para retornar alunos com % concluído menor ou igual ao valor informado. Crie 25 alunos aleatórios, filtre os com `<= 50%` e ordene por ID.

```
class Student implements Comparable<Student> {
    private static int nextId = 1;
    private int id = nextId++;
    private String nome, curso;
    private double percentual;

    @Override
    public int compareTo(Student outro) {
        return Integer.compare(this.id, outro.id); // ordem natural: ID
    }

    // Comparator alternativo:
    public static Comparator<Student> POR_CURSO =
        Comparator.comparing(Student::getCurso);
}

class QueryList<T extends Comparable<T>> extends ArrayList<T> {
    // herda toda a API de ArrayList – sem campo extra!

    public QueryList<T> filtrarMenorQue(double pct) {
        return this.stream()
            .filter(item -> item instanceof LPASStudent lpa
                && lpa.getPercentual() <= pct)
            .collect(Collectors.toCollection(QueryList::new));
    }
}

// Uso:
QueryList<LPASStudent> lista = new QueryList<>();
// adicionar 25 alunos...
lista.sort(Comparator.naturalOrder()); // por ID (Comparable)
QueryList<LPASStudent> atrasados = lista.filtrarMenorQue(50.0);
```

Mini-Project C

Repository Pattern Genérico

Crie a interface `Repository` com `findById`, `findAll`, `save` e `delete`. Implemente `InMemoryRepository` que usa um `Map` como armazenamento. Crie `AlunoRepository` extends `InMemoryRepository`. Demonstre que o mesmo código de `Repository` funciona para qualquer entidade.

```
interface Repository<T, ID> {
    Optional<T> findById(ID id);
    List<T> findAll();
    void save(T entity);
    void delete(ID id);
}

class InMemoryRepository<T, ID> implements Repository<T, ID> {
    private final Map<ID, T> store = new HashMap<>();
    private java.util.function.Function<T, ID> keyExtractor;

    public InMemoryRepository(java.util.function.Function<T, ID> ke) {
        this.keyExtractor = ke;
    }
    @Override public void save(T entity) {
        store.put(keyExtractor.apply(entity), entity);
    }
    @Override public Optional<T> findById(ID id) {
        return Optional.ofNullable(store.get(id));
    }
    @Override public List<T> findAll() {
        return new ArrayList<>(store.values());
    }
    @Override public void delete(ID id) { store.remove(id); }
}

// Uso — mesmo código, tipos diferentes:
var alunoRepo = new InMemoryRepository<Aluno, Integer>(Aluno::getId);
var cursoRepo = new InMemoryRepository<Curso, String>(Curso::getCodigo);
```

Resumo Final

Generics	Eliminam casting manual e garantem type-safety em compile-time. Custo zero em runtime.
Classe genérica	class Box — T é substituído pelo tipo concreto na compilação.
Convenções	T=Type, E=Element, K=Key, V=Value. Diamond <> infere o tipo.
Raw Types	Evite — existem apenas por retrocompatibilidade. Geram unchecked warnings.
Upper Bound	— restringe o tipo e libera acesso a métodos do bound.
Múltiplos Bounds	& separa bounds. Classe primeiro, depois interfaces. Máximo 1 classe.
Covariância	List NÃO é subtipo de List. Use wildcards para resolver.
Wildcards	irrestrito. para leitura. para escrita. PECS.

Type Erasure	Generics removidos do bytecode. $T \rightarrow \text{Object}$ (ou bound). Sem reflexão genérica em runtime.
Comparable	Define ordem natural. <code>compareTo()</code> retorna negativo/zero/positivo.
Comparator	Ordem alternativa. <code>Comparator.comparing()</code> + <code>thenComparing()</code> em Java 8+.